

TypeScript Constructors

The Complete Beginner-to-Expert Guide

■ Interview Ready

■■ Framework Focused

■ Playwright Examples

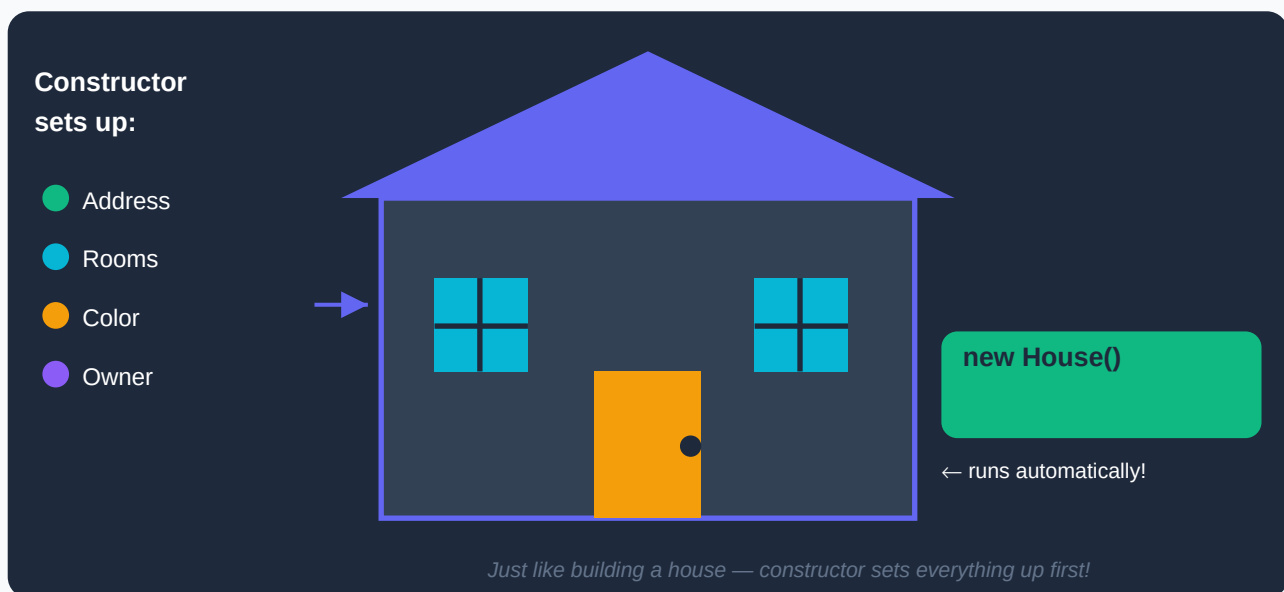
#	Topic	Page
1	What is a Constructor? (Real-world analogy)	2
2	Syntax & How it works	3
3	Type 1 — Default Constructor	4
4	Type 2 — Parameterized Constructor	5
5	Type 3 — Default Parameters Constructor	6
6	Type 4 — Constructor Overloading	7
7	Type 5 — Parameter Properties Shorthand	8
8	Type 6 — Access Modifiers in Constructor	9
9	Type 7 — Abstract Class Constructor	10
10	Type 8 — Private Constructor (Singleton)	11
11	super() — The Parent Caller	12
12	Constructors in Playwright POM	13
13	Interview Cheat Sheet + One-liners	14

Chapter 1

What is a Constructor?

The House Analogy ■

Before anyone can live in a house, a builder must **set it up first** — assign an address, decide the number of rooms, paint it a colour, and name an owner. A **constructor** does exactly the same thing for a TypeScript **class**: it runs automatically the moment you create an object and makes sure everything is ready to use.



■ Simple Definition

A constructor is a special method inside a class that runs automatically when you write `new ClassName()`. Its job is to set up (initialise) all the properties of the object so it is ready to use.

Key Rules — Remember These!

- **Always named** — constructor — no other name is allowed.
- **Runs automatically** — You never call it manually — `new ClassName()` triggers it.
- 1 ■ **Only one per class** — TypeScript allows only one constructor implementation.
- **Returns the object** — You never write `return` — the new object is returned automatically.

How it Looks in Code

```
class ClassName {  
  // Step 1: declare properties  
  propertyName: type;  
  
  // Step 2: constructor sets them up  
  constructor(/* parameters */) {
```

```
    this.propertyName = value;    // 'this' = the object being created
  }
  // Step 3: methods use those properties
  someMethod() { ... }
}
// Usage
const obj = new ClassName();    // constructor runs here automatically ■
```

The Flow — What Happens When You Write new?



Chapter 2

All 8 Types of Constructors

Type 1

Default Constructor — No Parameters

The simplest form. No parameters needed — values are **hardcoded** inside the constructor. Use it when every object of that class should start with the same defaults.

■ Analogy

Like a furniture shop that always ships a standard sofa — same colour, same size. You don't choose anything; it just comes as-is.

```
class Browser {
  name: string;
  isHeadless: boolean;
  constructor() {                // ← no parameters
    this.name      = 'Chromium';  // hardcoded default
    this.isHeadless = false;
  }
}

const b = new Browser();          // no arguments needed
console.log(b.name);              // Chromium
console.log(b.isHeadless);        // false
```

■ In Playwright

```
class TestConfig {
  timeout: number;
  retries: number;
  constructor() {
    this.timeout = 30000; // 30 seconds default
    this.retries = 2;
  }
}

const cfg = new TestConfig();    // instantly usable ■
```

Type 2

Parameterized Constructor — Required Parameters

Parameters are **required**. You pass values when creating the object. This is the **most common type** in real projects, especially in Playwright's Page Object Model.

■ Analogy

Like ordering a custom cake — you must tell the bakery the flavour, size, and message. Without those details they can't make it.

```
class User {
  username: string;
  role: string;

  constructor(username: string, role: string) { // ← required params
    this.username = username;
    this.role     = role;
  }
}

const admin = new User('Ravi', 'admin');
const guest = new User('Priya', 'viewer');
console.log(admin.username); // Ravi
console.log(guest.role);    // viewer
```

■ In Playwright — LoginPage POM

```
import { Page, Locator } from '@playwright/test';

class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) { // Page object is REQUIRED
    this.page = page;
    this.usernameInput = page.locator('#username');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('#loginBtn');
  }

  async login(user: string, pass: string) {
    await this.usernameInput.fill(user);
    await this.passwordInput.fill(pass);
    await this.loginButton.click();
  }
}

// In test:
const loginPage = new LoginPage(page); // constructor runs here!
await loginPage.login('admin', 'pass123');
```

Type 3

Default Parameters — Optional with Fallback Values

Parameters are **optional**. If you don't pass them, the default values are used. Great for configuration classes where you want sensible defaults but also allow overrides.

■ Analogy

Like a coffee machine with presets — by default it brews a medium espresso, but you can override the size or strength if you want something different.

```
class TestEnvironment {
  baseUrl: string;
  browser: string;
  headless: boolean;

  constructor(
    baseUrl: string = 'https://staging.myapp.com', // default
    browser: string = 'chromium',                  // default
    headless: boolean = true                       // default
  ) {
    this.baseUrl = baseUrl;
    this.browser = browser;
    this.headless = headless;
  }
}

const env1 = new TestEnvironment();
// baseUrl = https://staging.myapp.com, browser = chromium

const env2 = new TestEnvironment('https://prod.myapp.com');
// baseUrl overridden, rest still defaults

const env3 = new TestEnvironment('https://dev.myapp.com', 'firefox', false);
// everything overridden
```

Call	baseUrl	browser	headless
<code>new TestEnvironment()</code>	<code>https://staging.myapp.com</code>	<code>chromium</code>	<code>true</code>
<code>new TestEnvironment('https://prod.myapp.com')</code>	<code>https://prod.myapp.com</code>	<code>chromium</code>	<code>true</code>
<code>new TestEnvironment('https://dev.myapp.com', 'firefox', false)</code>	<code>https://dev.myapp.com</code>	<code>firefox</code>	<code>false</code>

Type 4

Constructor Overloading — Multiple Signatures

TypeScript lets you declare **multiple signatures** so the same class can be created in different ways. However, there is only **one implementation body** that handles all cases using if/else logic. (Unlike Java/C#, TypeScript does NOT support multiple constructor bodies.)

■ Analogy

Like booking a hotel — you can book by giving a date range, OR by giving a booking reference code. Two different inputs, same end result: a confirmed room.

```
class TestData {
  username: string;
  password: string;
  role: string;

  // Signature 1 – pass individual strings
  constructor(username: string, password: string, role: string);

  // Signature 2 – pass a single object
  constructor(data: { username: string; password: string; role: string });

  // ONE implementation that handles BOTH
  constructor(
    usernameOrData: string | { username: string; password: string; role: string },
    password?: string,
    role?: string
  ) {
    if (typeof usernameOrData === 'object') {
      // Called with an object
      this.username = usernameOrData.username;
      this.password = usernameOrData.password;
      this.role = usernameOrData.role;
    } else {
      // Called with individual strings
      this.username = usernameOrData;
      this.password = password!;
      this.role = role!;
    }
  }
}

// Both of these work!
const u1 = new TestData('Ravi', 'pass123', 'admin');
const u2 = new TestData({ username: 'Priya', password: 'pass456', role: 'viewer' });
```

This is a **TypeScript-only** shortcut. Instead of declaring properties at the top of the class AND assigning them in the constructor, you do **both in one line** by adding an access modifier directly in the constructor parameters.

■ Analogy

Like a flat-pack furniture set that self-assembles — instead of taking out each piece, reading the manual, and screwing it in separately, it snaps together in one step automatically.

■ Long Way (without shorthand)

```
class LoginPage {
  readonly page: Page;
  readonly url: string;
  constructor(page: Page, url: string) {
    this.page = page; // assign manually
    this.url = url; // assign manually
  }
}
```

■ Short Way (with shorthand)

```
class LoginPage {
  // No property declarations needed!
  constructor(
    readonly page: Page,
    readonly url: string
  ) {} // body is empty – TS handles it!
}
```

■ In Playwright — Real Project Pattern

```
class CheckoutPage {
  constructor(
    private readonly page: Page,
    private readonly baseUrl: string = 'https://myapp.com'
  ) {}
  // 'page' and 'baseUrl' are automatically properties – no extra code needed!
  async navigate() {
    await this.page.goto(this.baseUrl + '/checkout'); // directly usable ■
  }
}
```

Access modifiers control **who can read or change** a property. They are placed before the property type in the constructor. This is crucial for writing secure, maintainable code.

■ **readonly** Set once, never changed

■ **public** Everyone can access

■ **protected** This class + Child classes

■ **private** Only THIS class can access

```
class UserAccount {
  constructor(
    public   username: string, // ■ accessible EVERYWHERE
    private password: string, // ■ only INSIDE this class
    protected role: string,  // ■ this class + CHILD classes
    readonly id: number      // ■ set once, NEVER changed
  ) {}

  showPassword() { return this.password; } // ■ works (inside class)
}

const u = new UserAccount('Ravi', 'secret', 'admin', 101);
console.log(u.username); // ■ Ravi
console.log(u.password); // ■ Error – private!
console.log(u.role);     // ■ Error – protected!
console.log(u.id);       // ■ 101
u.id = 999;              // ■ Error – readonly!
```

Modifier	Same Class	Child Class	Outside Class	Can Reassign?
public	■ Yes	■ Yes	■ Yes	■ Yes
private	■ Yes	■ No	■ No	■ Yes (inside)
protected	■ Yes	■ Yes	■ No	■ Yes (inside)
readonly	■ Yes	■ Yes	■ Yes	■ No

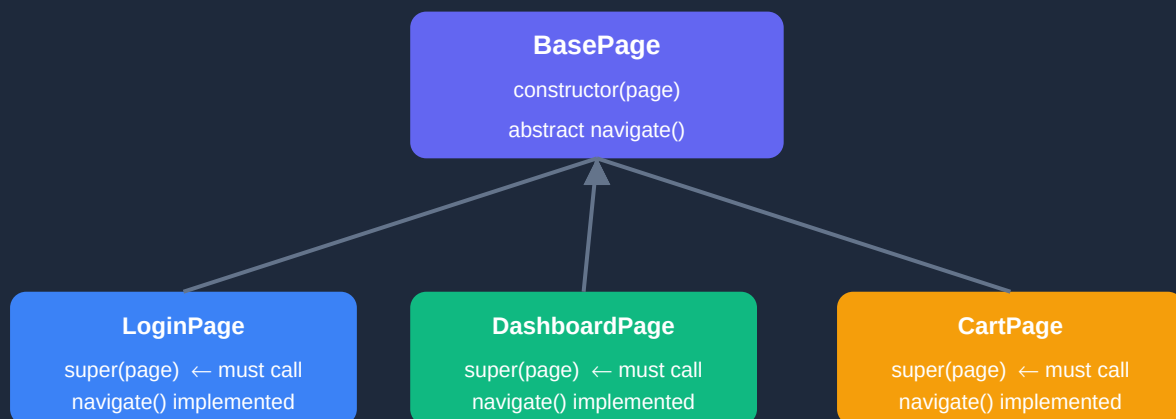
Type 7

Abstract Class Constructor — The Blueprint Pattern

An **abstract class** is a class that **cannot be used directly** — it must be extended by a child class. Think of it as a blueprint or template. The constructor in an abstract class sets up shared properties that all child classes will inherit.

■ Analogy

Like an architect's blueprint — you can't live in a blueprint, but every house built FROM it shares the same foundation, walls layout, and electrical plan. Each house then adds its own finishing touches.



All child classes MUST call super() — it runs BasePage constructor first

```
abstract class BasePage {
  constructor(protected readonly page: Page) {}
  abstract navigate(): Promise; // child MUST implement this
  async takeScreenshot(name: string) { // shared by ALL pages
    await this.page.screenshot({ path: `./screenshots/${name}.png` });
  }
}

// ■ Cannot do this:
const base = new BasePage(page); // Error – abstract class!

// ■ Must extend:
class LoginPage extends BasePage {
  constructor(page: Page) {
    super(page); // runs BasePage constructor
  }
  async navigate() { // implements the abstract method
    await this.page.goto('/login');
  }
}
```

Type 8

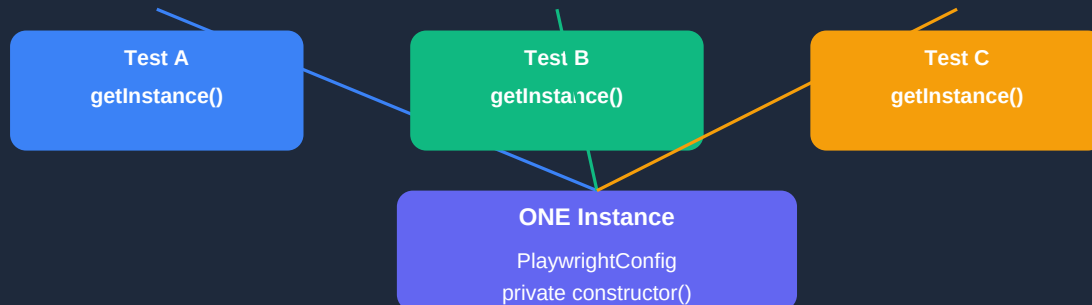
Private Constructor — Singleton Pattern

A **private constructor** prevents anyone from creating objects with `new`. Instead you provide a static `getInstance()` method that always returns **the same single object**. This guarantees only one instance ever exists — the

Singleton pattern.

■ Analogy

Like the one master key to a building — no matter who asks for it, they get the same key. You can't make copies; there's only ever one.



All calls return the SAME object — `getInstance()` always points to one instance

```
class PlaywrightConfig {
  private static instance: PlaywrightConfig; // holds the ONE object
  private baseUrl: string;
  private timeout: number;
  private constructor() { // ← private: blocks 'new'
    this.baseUrl = process.env.BASE_URL || 'https://myapp.com';
    this.timeout = Number(process.env.TIMEOUT) || 30000;
  }
  static getInstance(): PlaywrightConfig {
    if (!PlaywrightConfig.instance) {
      PlaywrightConfig.instance = new PlaywrightConfig(); // only once
    }
    return PlaywrightConfig.instance; // always the same object
  }
  getBaseUrl() { return this.baseUrl; }
  getTimeout() { return this.timeout; }
}

// ■ Error: constructor is private
const c = new PlaywrightConfig();

// ■ Correct way:
const cfg1 = PlaywrightConfig.getInstance();
const cfg2 = PlaywrightConfig.getInstance();
console.log(cfg1 === cfg2); // true — same object!
```

Chapter 3

super() — Calling the Parent Constructor

When a class **extends** another class (inheritance), the child class constructor **MUST** call `super()` before it can use this. Think of `super()` as pressing the 'Start' button on the parent class to get it ready before the child adds its own setup.

■ ■ Golden Rule

In any child class constructor, `super()` must ALWAYS be the FIRST line before you access this. Forgetting it causes a runtime error.

```
class Animal {
  name: string;
  sound: string;
  constructor(name: string, sound: string) {
    this.name = name;
    this.sound = sound;
    console.log('Animal constructor ran!');
  }
}

class Dog extends Animal {
  breed: string;
  constructor(name: string, breed: string) {
    super(name, 'Woof'); // ← MUST be first! runs Animal constructor
    this.breed = breed;  // ← now we can use this
    console.log('Dog constructor ran!');
  }
}

const dog = new Dog('Buddy', 'Labrador');
// Console: Animal constructor ran!
// Console: Dog constructor ran!
console.log(dog.name); // Buddy (from Animal)
console.log(dog.sound); // Woof (from Animal via super)
console.log(dog.breed); // Labrador (from Dog)
```

■ In Playwright POM — Full Chain

```
// BasePage – shared setup for all pages
abstract class BasePage {
  constructor(protected readonly page: Page) {
    console.log('BasePage ready');
  }
  async waitForLoad() {
```

```

        await this.page.waitForLoadState('networkidle');
    }
}

// LoginPage - adds its own locators
class LoginPage extends BasePage {
    readonly loginBtn: Locator;
    constructor(page: Page) {
        super(page); // BasePage constructor runs first
        this.loginBtn = page.locator('#loginBtn'); // then LoginPage setup
    }
}

// DashboardPage - adds its own locators
class DashboardPage extends BasePage {
    readonly welcomeText: Locator;
    constructor(page: Page) {
        super(page);
        this.welcomeText = page.locator('.welcome-msg');
    }
}

```

Chapter 4

Constructors in Playwright — Full POM Example

Below is a production-ready Page Object Model structure that uses **all the constructor concepts** you have learned. Study this pattern — it is what interviewers expect you to implement!

BasePage.ts

```
import { Page } from '@playwright/test';

export abstract class BasePage {
  // Parameter Properties shorthand (Type 5)
  constructor(protected readonly page: Page) {}

  // All pages MUST implement navigate()
  abstract navigate(): Promise;

  async takeScreenshot(name: string) {
    await this.page.screenshot({ path: `./screenshots/${name}.png` });
  }

  async getTitle(): Promise {
    return this.page.title();
  }
}
```

LoginPage.ts

```
import { Page, Locator } from '@playwright/test';
import { BasePage } from './BasePage';

export class LoginPage extends BasePage {
  private readonly usernameInput: Locator;
  private readonly passwordInput: Locator;
  private readonly loginButton: Locator;
  private readonly errorMessage: Locator;

  constructor(page: Page) {
    super(page); // Type 7: abstract class
    this.usernameInput = page.locator('#username');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('#loginBtn');
    this.errorMessage = page.locator('.error-msg');
  }

  async navigate() {
    await this.page.goto('/login');
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}
```

```
    }  
    async getError(): Promise {  
        return this.errorMessage.textContent();  
    }  
}
```

login.spec.ts — The Test File

```
import { test, expect } from '@playwright/test';  
import { LoginPage } from '../pages/LoginPage';  
  
test('valid login', async ({ page }) => {  
    const loginPage = new LoginPage(page); // Parameterized constructor (Type 2)  
    await loginPage.navigate();  
    await loginPage.login('admin', 'pass123');  
    await expect(page).toHaveURL('/dashboard');  
});  
  
test('invalid login shows error', async ({ page }) => {  
    const loginPage = new LoginPage(page);  
    await loginPage.navigate();  
    await loginPage.login('wrong', 'wrong');  
    const error = await loginPage.getError();  
    expect(error).toBe('Invalid credentials');  
});
```

Chapter 5

Interview Cheat Sheet ■

Q: What is a constructor?

A: A special method that runs automatically when you create an object using 'new'. It sets up (initialises) the object's properties.

Q: How many types of constructors are there in TypeScript?

A: 8 types: Default, Parameterized, Default Parameters, Overloading, Parameter Properties Shorthand, Access Modifiers, Abstract Class, and Private (Singleton).

Q: Can a TypeScript class have multiple constructors?

A: No. Only one constructor implementation. Multiple signatures are allowed for overloading, but they funnel into one body.

Q: What is super() and why is it needed?

A: super() calls the parent class constructor. It must be the first line in a child class constructor before using 'this'.

Q: What is the most common constructor type in Playwright?

A: Parameterized constructor — passing 'page: Page' in the Page Object Model to initialise the page and all locators.

Q: What is Parameter Properties Shorthand?

A: A TypeScript feature where adding an access modifier in the constructor parameter (e.g. private readonly) auto-creates and assigns the property — no extra code needed.

Q: What does readonly do in a constructor?

A: It makes the property settable only once (during construction) and prevents any reassignment afterwards.

Q: What is the Singleton pattern and why use it?

A: Using a private constructor + static getInstance() to ensure only one instance of a class ever exists. Used for shared config, DB connections, or test helpers.

Q: What is an abstract class constructor used for?

A: To define shared setup code for a base class that child classes must extend. The abstract class itself cannot be instantiated directly.

Q: Difference between private and protected?

A: 'private' is accessible only inside the same class. 'protected' is accessible inside the same class AND any class that extends it.

Chapter 6

Quick Reference — All 8 Types at a Glance

#	Type	Syntax Hint	Best Used When	Playwright Use
1	Default	constructor() {}	All objects same defaults	TestConfig defaults
2	Parameterized	constructor(x: T) {}	Values differ per object	POM page classes
3	Default Params	constructor(x = val) {}	Optional config overrides	ApiHelper with defaults
4	Overloading	Multiple signatures + one body	Multiple creation styles needed	PageFactory flexible init
5	Param Properties	constructor(private x: T) {}	Shorthand, clean code	Modern POM classes
6	Access Modifiers	public/private/protected/readonly	Encapsulation & security	All POM locators
7	Abstract Class	abstract class + super()	Shared base for all pages	BasePage pattern
8	Private Singleton	private constructor + getInstanceOf	Shared instance needed once	Config/DB helpers

The Master One-Liner

■ Say This in Your Interview

"In TypeScript, constructors come in 8 forms — Default, Parameterized, Default Parameters, Overloading, Parameter Properties Shorthand, Access Modifiers, Abstract Class Constructor, and Private Constructor for Singleton. In Playwright POM, we use Parameterized constructors with readonly access modifiers and shorthand Parameter Properties to cleanly initialise the page object and locators. The Abstract BasePage pattern enforces consistency across all page classes, and the Singleton pattern is ideal for shared configuration objects."

Mental Map — Constructor Types

CONSTRUCTORS IN TYPESCRIPT

-
- 1. Default → constructor() {}
- 2. Parameterized → constructor(page: Page) {} ← MOST USED in Playwright
- 3. Default Parameters → constructor(url = 'https://...') {}
- 4. Overloading → multiple signatures, one body
- 5. Param Properties → constructor(private readonly page: Page) {}
- 6. Access Modifiers → public / private / protected / readonly
- 7. Abstract Class → abstract class Base { constructor() {} }
- 8. Private Constructor → private constructor() {} // Singleton

■ You Are Interview Ready!

You now understand all 8 types of constructors, access modifiers, `super()`, the Singleton pattern, abstract classes, and how to apply every concept in a real Playwright + TypeScript framework. Go build something awesome! ■